

Base Margin 부스팅 — 구현과 결과

날짜: 2026-08-21 실행: `python -m src.training.core.market_base_margin_boosting python -m src.training.staking.base_margin_profit_analysis`

무엇을 만들었나

세 방식의 구조 차이:

1단계 (현행)	$\eta = \ln(q) + \sum_k \beta_k \cdot x_k$	← 시장 위에 선형 항만
2단계	$\eta = \alpha \cdot \ln(q) + \gamma \cdot \ln(P_{\text{fund}})$	← 모든 피처를 γ 스칼라 하나로 압축
Base Margin	$\eta = \ln(q) + f_{\text{tree}}(x)$	← 시장 위에 비선형 함수 전체를

Base Margin은 부스팅의 초기값을 $\ln(q)$ 로 두는 방식이다. 트리가 "시장이 이미 맞힌 부분"을 다시 배울 이유가 없어서, 처음부터 **시장 예측의 오차만** 파고든다. XGBoost의 `base_margin`, LightGBM의 `init_score`에 해당한다.

직접 구현한 이유

이 환경은 `xgboost/lightgbm/catboost` 설치가 조직 네트워크 정책으로 차단돼 있고 (`round4_followup_20260820`에서 `tqdm` 같은 사소한 패키지도 동일하게 막히는 것으로 확인), `sklearn` 부스팅은 `base margin`을 지원하지 않는다. 그래서 **경주단위 조건부 로짓 손실에 대한 Newton 부스팅**을 직접 작성했다.

```
손실:    LL = Σ 경주 [ η_승자 - logsumexp(η) ]
1차 미분: g_i = y_i - p_i
2차 미분: h_i = p_i(1 - p_i)
윗 값:   w = Σg / (Σh + λ)      ← XGBoost와 같은 Newton 스텝
갱신:   η ← η + 1r · w
```

`sklearn DecisionTreeRegressor`는 트리 **구조 학습에만** 쓰고, 윗 값은 위 식으로 덮어썼다(회귀 트리 기본 윗 값은 단순 평균이라 2차 정보를 못 쓴다). 트리 개수는 학습창 내부의 시간순 뒤쪽 20%를 떼어 조기중단으로 정했고, 테스트 fold는 보지 않았다.

결과 1 — 정보량은 프로젝트 최고치

방식	ΔLL/경주	95% CI	판정
1단계 (선형)	+0.00491	[+0.00126, +0.00864]	확정
2단계 (γ 압축)	+0.00220	[+0.00044, +0.00396]	확정
Base Margin	+0.00739	[+0.00437, +0.01044]	확정

방식	ΔLL /경주	95% CI	판정
앵커 없음 (대조군)	-0.19455	[-0.21331, -0.17581]	불과

기존 최고인 1단계보다 **50% 높고**, CI 하한도 +0.00126 → +0.00437로 크게 올라갔다. 표본외 정보량 기준으로는 이 프로젝트에서 나온 가장 좋은 값이다.

대조군이 시장 앵커링의 기여를 직접 보여준다. 같은 부스터를 $\eta = 0$ 에서 시작하게 하면 ΔLL 이 -0.19455로 무너지다. 즉 트리가 자력으로 배운 것은 시장 배당을 그대로 쓰는 것보다 훨씬 나쁘다. 벤틀 철학("시장은 거의 옳다")이 코드 수준에서 확인된 셈이다.

fold별 트리 수는 2473개로 편차가 크다. ~~조기중단이 fold마다 다르게 걸렸다는 뜻이고, 학습 데이터가 쌓일수록 (fold 57) 트리를 더 많이 쓰는 경향이 보인다.~~

결과 2 — 그런데 수익으로는 오지 않았다

같은 정책자금 백테스트: 579건, 적중률 5.35%, ROI +2.57%, CI [-31.24%, +41.00%]. 1단계(327건, 13.15%, +32.69%)보다 나뉘었다.

적중률 5.35%는 정책이 룬샷 쪽으로 갔다는 뜻이다. 왜 그런지 배당구간별로 뜯어봤다.

배당구간별 edge-결과 상관

구간	n	1단계	Base Margin
1-3배	3,392	+0.0476	+0.0534
3-5배	4,214	+0.0500	+0.0559
5-8배	5,247	+0.0143	+0.0591
8-15배	7,857	+0.0660	+0.0319
15-30배	8,530	+0.0272	+0.0436
30배+	12,982	+0.0085	+0.0279

여기서 그림이 갈린다.

- 1단계는 정보가 8~15배 구간에 몰려 있다 (+0.0660, 다른 구간의 1.3~7.8배)
- Base Margin은 모든 구간에 고르게 퍼져 있다 (+0.028 ~ +0.059)

Base Margin은 총 정보량이 더 많지만 **얇게 퍼 발린 형태**다. 그리고 정작 실전에서 쓰는 8~15배 구간에서는 1단계가 두 배 이상 낮다.

같은 정책에서 직접 비교

배당 8~15배 · 옛지 상위 5%로 정책을 고정하고 모델만 바꿔 끼운 비교:

모델	벤틀수	적중률	평균배당	ROI	95% CI
1단계	395	14.94%	10.33	+47.47%	[+14.16%, +82.73%]

모델	베팅수	적중률	평균배당	ROI	95% CI
Base Margin	395	12.66%	9.96	+19.52%	[-9.48%, +52.70%]

이 표의 절대 수치는 낙관 편향이 있다. 8~15배·상위5%라는 정책은 과거에 이 데이터를 보고 선택된 것이다. 새로 탐색하지는 않았으므로 두 모델의 **상대비교**로는 유효하지만, +47.47%를 표본외 수익률 추정치로 읽으면 안 된다. 정책까지 제대로 잠근 원본 라운드의 값은 327건 +32.69% (CI가 0을 포함)이다.

상대비교로만 보면, 베팅 목적에서는 1단계가 Base Margin보다 낫다.

해석

정보량(ΔL)과 수익성(ROI)이 갈라진 이유에 대해 가능한 설명:

부스팅은 유연해서 **모든 구간에서** 시장을 조금씩 교정한다. 여기엔 인기마 구간(15배)도 포함되는데, 그 구간은 ~~사장이 아마 거의 정확하고 flat ROI가 15~20%다~~. 즉 그 교정은 로그우도는 올려주지만 **베팅할 수 없는 개선**이다. 20% 공제율을 넘길 만큼 크지 않기 때문이다.

반면 1단계는 선형 구조에 Bonferroni 선택까지 걸려 있어 fold당 14개 ~~피처만 쓴다~~. 그 소수 피처(clinic_30d_age)의 마가격이 ~~마침~~ 8~15배 구간에 몰려 있어서, 정보가 **베팅 가능한 곳에 집중**된다.

정리하면 — 더 많이 아는 것과 더 버는 것은 다르다. 공제율 장벽이 있는 시장에서는 **정보가 어디에 있느냐**가 정보의 총량만큼 중요해 보인다. 다만 이는 사후 해석이고, 표본이 늘거나 부스터 설정을 바꾸면 다르게 보일 수 있다.

정리

항목	결과
Base Margin 구현	작동함. 커스텀 Newton 부스팅으로 직접 구현
정보량 (ΔL)	+0.00739 — 프로젝트 최고 , 1단계 대비 +50%
시장 앵커링 기여	대조군 -0.19455로 직접 확인됨
수익성	개선 안 됨. 같은 정책에서 +19.52% vs 1단계 +47.47%
원인	정보가 전 구간에 얇게 퍼짐. 베팅 가능 구간(8~15배)에서는 1단계가 2배 이상 우위
실무 권고	베팅 모델은 1단계 유지

Base Margin 코드는 저장소에 남는다. 제주·부경 배당을 확보해 표본이 늘거나 피처가 추가되면 다시 비교해볼 여지가 있다 — 정보가 얇게 퍼지는 문제가 표본 부족 때문일 가능성도 배제할 수 없다.

산출물

파일	내용
src/training/core/market_base_margin_boosting.py	Base Margin 부스팅 (대조군 동시 실행)

파일	내용
src/training/staking/base_margin_profit_analysis.py	정보-수익 괴리 진단
outputs/reports/base_margin_boosting_20260821/	Δ LL 결과
outputs/reports/base_margin_profit_20260821/	구간별 edge 품질·용량곡선